

# **Assistente inteligente de transporte público**

## **1.1 Descrição do problema**

### **Qual é o problema que você deseja resolver?**

O problema é facilitar o acesso a informações atualizadas sobre o transporte público. Muitas vezes, horários, itinerários, tarifas e alterações temporárias ficam espalhados em diferentes sites, documentos e comunicados. O sistema permitirá que o usuário faça perguntas em linguagem natural e receba uma resposta rápida baseada em informações oficiais.

Para descobrir informações sobre ônibus, metrô ou outros transportes públicos, o usuário muitas vezes precisa procurar em vários lugares diferentes. Os horários podem estar disponíveis em um site ou aplicativo, enquanto os itinerários podem aparecer em mapas, PDFs ou páginas específicas. As tarifas podem ser divulgadas em comunicados ou no site da empresa, e alterações temporárias, como mudanças de rota, obras, feriados ou interrupções, podem ser informadas separadamente. Isso torna a busca demorada e confusa, principalmente quando a informação mais recente está em um lugar diferente da informação antiga. Como solução, a ideia é criar um sistema em que o usuário possa simplesmente fazer uma pergunta usando linguagem natural, como se estivesse conversando com alguém, e receber rapidamente uma resposta baseada em informações oficiais e atualizadas.

### **Quem utilizaria a aplicação?**

A aplicação seria utilizada principalmente por passageiros de transporte público, como estudantes, trabalhadores e moradores da cidade que utilizam ônibus no dia a dia. O usuário teria pouco ou nenhum conhecimento técnico e faria perguntas de maneira simples, como se estivesse conversando com um atendente.

### **Que tipo de informação o usuário gostaria de consultar?**

O usuário poderia consultar:

- horários das linhas;
- itinerários;
- pontos de parada;
- tarifas;
- integrações entre linhas;
- funcionamento em domingos e feriados;
- alterações temporárias de trajetos;
- interrupções ou mudanças no serviço.

### **De onde vêm essas informações?**

As informações seriam obtidas principalmente de fontes oficiais do órgão responsável pelo transporte público, como:

- site oficial;
- documentos em PDF;
- tabelas de horários;
- comunicados;
- regulamentos;
- informações sobre tarifas;
- APIs ou bancos de dados disponibilizados pelo sistema de transporte.

### **Por que utilizar um LLM sozinho não seria suficiente?**

Um LLM, por si só, não seria a melhor opção para esse tipo de sistema, pois suas informações podem estar desatualizadas, especialmente em relação a horários, tarifas e trajetos. No transporte público, esses dados podem mudar constantemente devido a obras, feriados, alterações de preços ou mudanças nas linhas. Além disso, existe o risco de o modelo fornecer uma resposta convincente, mas incorreta. Para evitar esse problema, o RAG permite que o sistema busque informações em fontes oficiais antes de responder. Assim, a resposta é baseada em dados mais recentes e confiáveis, além de possibilitar que o usuário saiba de onde as informações foram obtidas.

### **Como o usuário vai utilizar o sistema? (API, aplicativo, interface web?)**

O usuário utilizaria o sistema por meio de um aplicativo ou uma interface web de conversa. Ele faria uma pergunta normalmente, o sistema buscaria as informações relevantes nas fontes oficiais e o LLM utilizaria essas informações para formular uma resposta clara.

### **Três perguntas reais que um usuário faria**

Qual ônibus eu posso pegar para chegar ao centro no domingo de manhã?

Tem ônibus para o shopping no sábado?

A linha 205 funciona aos domingos?

## **1.2 Por que RAG?**

### **Por que RAG é adequado para esse problema?**

O RAG é adequado porque permite que o sistema consulte informações atualizadas antes de responder ao usuário. Como os dados do transporte público podem mudar frequentemente, o RAG reduz o risco de o sistema fornecer informações antigas ou incorretas.

### **Que tipo de conhecimento precisa ser fornecido ao modelo?**

O modelo precisa ter acesso a informações como horários dos ônibus e metrô, itinerários, tarifas, pontos de parada, dias de funcionamento e comunicados sobre alterações temporárias, como mudanças de rota, obras, feriados e interrupções.

### **Esse conhecimento muda com que frequência? (diariamente, mensalmente, quase nunca?)**

Essas informações não possuem uma frequência única de atualização. Horários e trajetos das linhas podem sofrer alterações frequentes, inclusive de um dia para o outro, devido a obras, eventos ou outros imprevistos. Já tarifas e informações gerais costumam ser modificadas com menos frequência, podendo permanecer iguais por meses. Por isso, o sistema deve utilizar fontes confiáveis e que permitam atualizações constantes, garantindo que as respostas estejam sempre baseadas nas informações mais recentes.

### **Existe necessidade de utilizar documentos privados ou específicos da organização?**

Sim. O sistema pode utilizar documentos específicos dos órgãos ou empresas responsáveis pelo transporte, como tabelas de horários, mapas, comunicados oficiais, documentos em PDF e avisos sobre alterações nas linhas. Esses documentos podem servir como fontes para o RAG.

### **Que problemas poderiam ocorrer se o LLM respondesse apenas com seu conhecimento pré-treinado? Dê um exemplo concreto de resposta errada que ele daria no seu cenário.**

Como o LLM não possui acesso garantido aos dados atuais, ele pode fornecer informações incorretas ou desatualizadas.

#### **Exemplo:**

**Pergunta:** "Qual é o valor atual da passagem?"

**Resposta do LLM:** "A passagem custa R\$ 4,50."

**Problema:** A tarifa atual já foi alterada para R\$ 5,00.

### **1.3 Limitações - quando RAG não é a resposta**

Apesar de o RAG ser útil para buscar e responder perguntas com base em documentos, existem situações em que outras abordagens são mais adequadas. A escolha da técnica depende do tipo de informação que o usuário deseja consultar e da forma como esses dados estão armazenados.

#### **Busca tradicional por palavra-chave**

A busca por palavras-chave é suficiente para encontrar termos específicos em documentos de forma rápida, sem precisar utilizar um LLM.

#### **Banco de dados estruturado e consultas SQL**

Um banco de dados seria mais adequado para informações organizadas em tabelas, como linhas, horários, tarifas, pontos de parada e dias de funcionamento. Nesse caso, consultas SQL conseguem filtrar, comparar, contar e ordenar dados com maior

precisão.

#### Utilização direta de uma API

Se a empresa tiver uma **API com dados em tempo real**, é melhor consultá-la diretamente, pois ela fornece informações mais precisas e atualizadas, como a localização de um ônibus ou seu próximo horário.

**Existe alguma pergunta, dentro do seu próprio cenário, que RAG responderia mal e um banco de dados relacional responderia bem? Qual, e por quê?**

Quantas linhas de ônibus possuem mais de 10 horários entre 6h e 9h?

Essa pergunta envolve contagem e comparação de dados estruturados. Um banco de dados relacional seria mais adequado, pois permite realizar filtros, agrupamentos e contagens com SQL. O RAG poderia encontrar documentos relacionados aos horários, mas teria mais dificuldade para garantir que todos os dados fossem considerados corretamente.

**O que aconteceria se a pergunta do usuário exigisse contar, somar ou ordenar informação espalhada por muitos documentos?**

Se fosse necessário contar, somar ou ordenar informações de vários documentos, o RAG poderia apresentar dificuldades, pois nem sempre recuperaria todos os dados necessários. Nesse caso, seria melhor organizar essas informações em um banco de dados e usar SQL para realizar os cálculos e a ordenação com maior precisão.

## Parte 2 - Organização dos documentos

**Quais tipos de arquivo existirão? (PDF, DOCX, HTML, Markdown, páginas web, planilhas, imagens, áudios, vídeos, outros)**

**PDF:** tabelas de horários, comunicados, mapas e documentos oficiais.

**HTML/páginas web:** informações publicadas nos sites das empresas e órgãos de transporte.

**Planilhas (XLSX/CSV):** horários, tarifas, linhas e pontos de parada.

**DOCX:** comunicados e documentos administrativos, quando existirem.

Imagens: mapas e avisos que não estejam disponíveis em formato textual.

**Qual o volume aproximado? (dezenas, centenas, milhares de documentos?)**

Inicialmente, o sistema poderia trabalhar com aproximadamente **500 documentos**, podendo chegar a **5.000 ou mais** conforme a quantidade de linhas, empresas e períodos de atualização.

**Qual o tamanho típico de cada documento? (Páginas, kbs)**

Comunicados

1 a 3 páginas.

|                                         |                                                           |
|-----------------------------------------|-----------------------------------------------------------|
| Tabelas e documentos de horários        | 5 a 30 páginas.                                           |
| Relatórios ou documentos mais completos | 30 a 100 páginas.                                         |
| Planilhas                               | 100 a 10.000 registros, dependendo do tipo de informação. |

### Com que frequência novos documentos entram? Documentos antigos são atualizados ou substituídos?

As atualizações dependem do tipo de informação. Comunicados sobre alterações de linhas, obras e interrupções podem surgir diariamente, enquanto tabelas de horários podem ser atualizadas semanalmente ou mensalmente. Tarifas normalmente mudam com menor frequência.

Documentos antigos não deveriam simplesmente ser apagados. Eles devem ser arquivados e identificados como versões antigas, enquanto somente a versão atual deve ser utilizada como fonte principal para responder às perguntas.

### Organização das pastas

```
documentos/
├── horarios/
├── itinerarios/
├── tarifas/
├── comunicados/
├── pontos_de_parada/
├── mapas/
└── arquivo/
```

A divisão dos documentos foi feita considerando os principais tipos de informação que um usuário procura quando utiliza o transporte público. Em vez de organizar os arquivos apenas pelo formato, como PDF, Excel ou Word, eles são separados pelo conteúdo que apresentam. Isso facilita tanto a organização quanto a busca das informações posteriormente.

|                     |                                                                                    |
|---------------------|------------------------------------------------------------------------------------|
| <b>horarios/</b>    | Reúne documentos relacionados aos horários de funcionamento e passagem das linhas. |
| <b>itinerarios/</b> | Contém informações sobre os trajetos e as rotas realizadas por cada linha.         |
| <b>tarifas/</b>     | Armazena informações sobre preços das passagens e possíveis alterações de valores. |

|                          |                                                                                                            |
|--------------------------|------------------------------------------------------------------------------------------------------------|
| <b>comunicados/</b>      | Reúne avisos oficiais sobre mudanças temporárias, como obras, alterações de rota, feriados e interrupções. |
| <b>pontos_de_parada/</b> | Contém informações sobre os locais onde os veículos realizam embarque e desembarque.                       |
| <b>mapas/</b>            | Armazena mapas das linhas, regiões atendidas e possíveis representações das rotas.                         |

### Existe documento que não deve entrar na base? (informação sigilosa, dado pessoal, versão obsoleta) Como você impediria a entrada?

Nem todos os arquivos devem ser disponibilizados ao sistema. Documentos que contenham dados pessoais, informações sigilosas, documentos internos sem autorização ou versões completamente obsoletas não devem ser utilizados como fonte de resposta.

Para evitar a entrada desses arquivos, poderia existir uma etapa de validação antes da indexação. Nessa etapa, os documentos seriam classificados de acordo com seu tipo, data, origem e nível de acesso. Apenas documentos autorizados e relevantes seriam adicionados à base do RAG.

### Como você lidaria com versões do mesmo documento? Se a política de férias mudou em 2026, o sistema pode recuperar a versão de 2024 e responder errado.

É importante controlar as versões para evitar que o sistema utilize informações antigas.

Cada documento poderia possuir metadados como:

- nome do documento;
- data de publicação;
- data de atualização;
- versão;
- status: atual ou arquivado;
- fonte oficial.

## Parte 3 - Pipeline de ingestão

```

Documentos
↓
Extração
↓
Limpeza / Normalização
↓
Metadados
↓
Chunking / Splitting

```

↓  
Embeddings  
↓  
Banco Vetorial

### Como o texto seria extraído?

Primeiro, o sistema identifica o tipo de arquivo recebido. Dependendo do formato, será utilizado um método diferente de extração.

|                 |                                                                                                                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PDF             | Extração do texto e das tabelas.                                                                                                                                                             |
| DOCX            | Leitura dos dados das planilhas.                                                                                                                                                             |
| HTML            | Extração do conteúdo relevante da página, removendo menus e elementos que não fazem parte da informação.                                                                                     |
| Imagens         | Utilização de OCR para transformar o conteúdo da imagem em texto.                                                                                                                            |
| Áudios e vídeos | Quando necessário, o conteúdo de áudios e vídeos pode ser convertido em texto por meio de ferramentas de transcrição, permitindo que essas informações também sejam utilizadas pelo sistema. |

### Como você trataria PDFs com texto selecionável?

Quando o PDF possui uma camada de texto, o sistema pode extrair diretamente esse conteúdo. Por exemplo, um documento oficial poderia conter:

Linha 204 – Segunda a sexta  
06:00 | 06:30 | 07:00 | 07:30

O sistema extrairia essas informações e posteriormente organizaria o conteúdo para ser utilizado pelo RAG.

### E PDFs digitalizados (imagem escaneada, sem camada de texto)?

Um problema ocorre quando o PDF é apenas uma imagem escaneada e não possui texto selecionável. Nesse caso, a extração tradicional não funcionaria. Para resolver isso, seria utilizado **OCR (Reconhecimento Óptico de Caracteres)**. O OCR analisa a imagem e transforma os caracteres encontrados em texto.

### Como trataria tabelas? (é importante manter?)

As tabelas são muito importantes nesse cenário e não devem simplesmente ser transformadas em texto desorganizado.

Se a estrutura for perdida durante a extração, o sistema poderia interpretar incorretamente os dados. Por isso, as tabelas poderiam ser convertidas para um formato estruturado, como CSV ou JSON, além de manter uma representação textual para o RAG.

### **Como tratar imagens?**

As imagens não devem ser descartadas automaticamente.

No transporte público, uma imagem pode conter informações importantes, como:

- Mapas de itinerários;
- Mapas de linhas;
- Avisos de alteração;
- Placas e informações de pontos;
- Diagramas de trajetos.

### **Como trataria documentos multimodais?(multimodais = texto + imagem, audio + video, texto + video e etc)**

Documentos multimodais, como PDFs que possuem textos, imagens, mapas ou tabelas, precisam ter seus diferentes tipos de conteúdo tratados de maneira adequada.

Por exemplo, um PDF pode apresentar um texto explicando uma alteração em determinada linha e, ao lado, um mapa mostrando o novo trajeto. Nesse caso, apenas extrair o texto não seria suficiente, pois o mapa também contém informações importantes para compreender a alteração.

Para áudios e vídeos, quando forem relevantes para o sistema, pode ser realizada uma transcrição do conteúdo. O texto transcrito pode ser armazenado juntamente com os metadados do arquivo original, como nome, data, origem e tipo de conteúdo.

Dessa forma, o sistema consegue aproveitar diferentes formatos de informação e fornecer respostas mais completas e precisas.

**Explique quais problemas podem surgir durante a extração. Se você já enfrentou algum deles nas atividades anteriores, cite o caso concreto.**

```
PDF com texto desorganizado;  
tabelas que perdem sua estrutura;  
páginas escaneadas sem camada de texto;  
erros de OCR;  
caracteres especiais ou acentos interpretados incorretamente;  
informações importantes presentes somente em imagens;  
cabeçalhos e rodapés sendo repetidos em todas as páginas.
```

## **3.2 Limpeza e normalização**

**O que precisa ser removido? (cabeçalhos e rodapés repetidos, numeração de página, marcas d'água, sumário, referências)**

## O que seria removido?

Cabeçalhos repetidos;  
rodapés repetidos;  
números das páginas;  
menus de páginas web;  
propagandas;  
espaços excessivos;  
quebras de linha desnecessárias;  
elementos de navegação;  
conteúdo duplicado.

## O que precisa ser padronizado? (acentuação, quebras de linha, espaçamento, codificação)

Espaços duplicados;  
quebras de linha;  
caracteres estranhos;  
codificação dos caracteres;  
acentuação quando houver problemas de codificação;  
formatos de datas;  
formatos de horários.

## Que informação você corre o risco de perder ao limpar demais?

É importante não fazer uma limpeza exagerada.

Informações como nome da linha, número do ônibus, horários, datas, destinos, tarifas e condições de funcionamento precisam ser preservadas.

Se a limpeza transformar uma tabela de horários em um texto sem relação entre linha, horário e destino, informações importantes poderão ser perdidas.

## 3.3 Frequência de ingestão

### O pipeline roda uma vez, sob demanda, ou de forma agendada? Com que frequência chegam novos documentos?

O pipeline seria executado de forma agendada e também sob demanda. Como informações de transporte podem sofrer alterações frequentes, o sistema poderia verificar diariamente se existem novos documentos ou atualizações. Em casos urgentes, como uma mudança de rota ou interrupção de uma linha, o documento poderia ser processado imediatamente.

### Quando um documento é atualizado, você reprocessa só ele ou a base inteira? Como sabe qual reprocessar?

Seria reproprocessado somente o documento que foi alterado, evitando processar toda a base novamente. Para identificar mudanças, o sistema poderia verificar a data de atualização, versão do documento ou uma identificação do conteúdo. Quando uma nova

versão fosse encontrada, a versão antiga seria marcada como obsoleta e a nova seria processada e adicionada ao banco vetorial.

## Parte 4 - Metadados

### 4.1 Metadados do documento

```
{
  "document_id": "doc-204-2026-03",
  "title": "Horários da Linha 204",
  "source": "Empresa de Transporte",
  "document_type": "horarios",
  "created_at": "2026-08-10",
  "updated_at": "2026-08-18",
  "category": "linha_204",
  "version": "3",
  "status": "atual"
}
```

#### Por que cada metadado é importante?

|               |                                                                                                              |
|---------------|--------------------------------------------------------------------------------------------------------------|
| document_id   | Identifica cada documento de forma única e permite relacionar os chunks ao documento original.               |
| title         | Informa o nome do documento e facilita sua identificação.                                                    |
| source        | Indica de onde a informação foi obtida. É importante para verificar se a fonte é oficial.                    |
| document_type | Identifica o tipo de informação, como horários, tarifas, itinerários ou comunicados.                         |
| created_at    | Mostra quando o documento foi criado.                                                                        |
| updated_at    | Indica quando a informação foi atualizada pela última vez. É importante para priorizar informações recentes. |
| category      | Identifica a qual linha ou assunto o documento pertence, facilitando a aplicação de filtros.                 |
| version       | Permite diferenciar versões diferentes do mesmo documento.                                                   |
| status        | Indica se o documento está atual ou arquivado, evitando que informações antigas sejam utilizadas.            |

## 4.2 Metadados do chunk

```
{
  "document_id": "doc-204-2026-03",
  "chunk_id": "doc-204-2026-03-05",
  "page": 5,
  "section": "Horários",
  "document_type": "horarios",
  "category": "linha_204",
  "text": "A linha 204 possui saídas às 06:00, 06:30 e 07:00."
}
```

### Por que cada metadado é importante?

|               |                                                                                     |
|---------------|-------------------------------------------------------------------------------------|
| document_id   | Permite saber de qual documento o chunk foi retirado.                               |
| chunk_id      | Identifica cada trecho individualmente dentro da base.                              |
| page          | Indica a página original onde a informação está localizada.                         |
| section       | Informa qual parte do documento contém o conteúdo, como "Horários" ou "Itinerário". |
| document_type | Permite filtrar chunks pelo tipo de informação.                                     |
| category      | Permite relacionar o chunk a uma linha específica.                                  |
| text          | Contém o conteúdo que será utilizado pelo sistema para responder à pergunta.        |

**Quais metadados você usaria para filtrar a busca? Dê um exemplo de pergunta em que o filtro é indispensável.**

```
category, document_type, status e updated_at.
```

### Exemplo:

```
Qual é o horário atual da linha 204?
```

### O sistema poderia filtrar por:

```
category = "linha_204"
document_type = "horarios"
status = "atual"
```

Esse filtro é importante porque existem documentos de várias linhas, diferentes tipos de informação e versões antigas. Sem o filtro, o sistema poderia recuperar um documento antigo ou relacionado a outra linha.

**Quais metadados você usaria para citar a fonte ao usuário? O que exatamente apareceria na tela junto da resposta?**

Seriam utilizados principalmente title, source, updated\_at e page.

Na tela, a resposta poderia aparecer assim:

A linha 204 possui saídas às 06:00, 06:30 e 07:00.  
Fonte: Horários da Linha 204 – Empresa de Transporte  
Atualizado em: 18/08/2026  
Página: 5

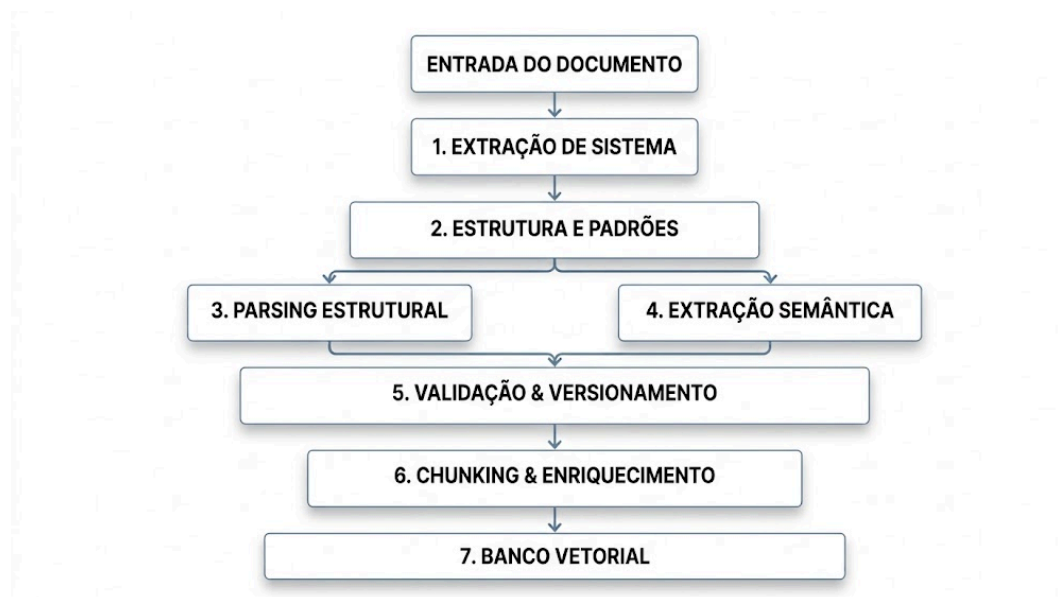
**Que metadado seria caríssimo de acrescentar depois que a base já estivesse indexada? Por quê?**

O document\_id e, principalmente, os metadados utilizados para filtragem, como category, document\_type, status e updated\_at, seriam caros de adicionar posteriormente.

Isso acontece porque esses dados precisam estar associados aos documentos e aos chunks já indexados. Se a base já estiver grande, seria necessário revisar e atualizar muitos registros e, em alguns casos, reprocessar os documentos e seus embeddings.

**Como você vai extrair esses metadados?**

Os metadados serão extraídos do próprio documento e de suas informações de origem. O sistema identificará automaticamente dados como nome, tipo, linha, data de atualização, categoria e fonte, organizando essas informações antes de armazená-las junto aos documentos e chunks.



## Parte 5 - Chunking / Splitting

### Qual estratégia de splitting você utilizaria?

Eu utilizaria um splitter recursivo, priorizando a divisão por seções e parágrafos. Quando uma seção fosse muito grande, ela seria dividida em partes menores por sentenças ou caracteres.

Essa estratégia é adequada porque mantém informações relacionadas juntas. Por exemplo, informações sobre os horários de uma mesma linha devem permanecer no mesmo chunk sempre que possível.

### Qual tamanho aproximado dos chunks?

Os chunks teriam aproximadamente 500 a 800 tokens, dependendo do documento. Esse tamanho permite que cada trecho tenha informações suficientes para responder a uma pergunta sem ficar grande demais.

### Utilizaria overlap? Quanto?

Sim. Utilizaria um overlap de aproximadamente 10% a 15% entre os chunks.

Isso ajuda a evitar que uma informação importante seja perdida quando uma frase ou explicação estiver exatamente na divisão entre dois chunks.

### A divisão seria por caracteres, palavras, sentenças, parágrafos ou seções?

|                                           |
|-------------------------------------------|
| Seção → Parágrafo → Sentença → Caracteres |
|-------------------------------------------|

A divisão seria feita principalmente por seções e parágrafos, mantendo assuntos relacionados no mesmo chunk. Caso uma seção fosse muito grande, ela poderia ser

dividida em sentenças menores. O objetivo é evitar separar informações que dependem umas das outras e manter o contexto necessário para o sistema responder corretamente.

### **Utilizaria um splitter recursivo?**

Sim. Utilizaria um splitter recursivo, pois ele permite dividir os documentos de forma gradual, tentando primeiro manter seções e parágrafos juntos. Se o trecho ainda ficar muito grande, o sistema pode dividi-lo em sentenças menores, preservando o máximo possível do contexto das informações.

### **Utilizaria uma estratégia específica para cada tipo de documento? Um contrato e uma transcrição de call center pedem o mesmo tratamento?**

Sim. Documentos diferentes precisam de tratamentos diferentes.

|             |                                                                           |
|-------------|---------------------------------------------------------------------------|
| Horários    | Manter a linha, horários e destinos relacionados.                         |
| Itinerários | Manter juntos os pontos e trajetos.                                       |
| Comunicados | Manter a informação sobre a alteração junto com a data e a linha afetada. |
| Tarifas     | Manter o valor, a data de validade e as condições da tarifa.              |
| Mapas       | Não dividir a imagem de forma que ela perca seu significado.              |

### **O que pode acontecer se os chunks forem muito pequenos?**

Chunks muito pequenos podem separar informações que deveriam estar juntas. No caso do transporte público, o horário, a linha e o destino poderiam acabar em chunks diferentes, dificultando a compreensão da informação.

### **O que pode acontecer se os chunks forem muito grandes?**

Chunks muito grandes podem dificultar a busca e a recuperação das informações específicas. Como o trecho contém muitos conteúdos diferentes, pode ser mais difícil encontrar exatamente o dado que o usuário procura.

## Como você trataria uma tabela na hora de dividir? Uma tabela cortada ao meio ainda significa alguma coisa? e uma imagem?

As tabelas seriam tratadas de forma diferente dos textos, pois é importante manter a relação entre linhas, horários e destinos. Sempre que possível, a tabela seria mantida inteira em um único chunk. Caso a tabela fosse muito grande, seria dividida por conjuntos de linhas, mantendo o cabeçalho da tabela em cada parte.

Imagens importantes, como **mapas de itinerários ou avisos**, não seriam descartadas. Elas seriam mantidas associadas ao trecho de texto correspondente. Quando necessário, poderiam ser analisadas por OCR ou por uma ferramenta de visão para extrair as informações presentes na imagem.

## Como saber se a sua escolha de chunking foi boa? Que evidência você juntaria para provar isso?

Para verificar se o método escolhido está funcionando corretamente, seria necessário realizar testes com perguntas relacionadas ao transporte público, como:

"Qual o horário da linha 204 para o centro?"  
"A linha 204 sofreu alguma alteração esta semana?"  
"Qual é a tarifa atual?"

Depois, seria analisado se o sistema consegue encontrar os chunks mais relevantes para cada pergunta.

## Parte 6 - Embeddings

|                                |                                                                                                                   |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Modelo escolhido               | Cohere Embed v4.0                                                                                                 |
| Dimensão do embedding          | 1536 por padrão, com opções de 256, 512 e 1024.                                                                   |
| Suporta português?             | Sim.                                                                                                              |
| É multilíngue?                 | Sim, suporta mais de 100 idiomas.                                                                                 |
| Tamanho máximo de entrada      | 128 mil tokens.                                                                                                   |
| É open source?                 | Não é um modelo open source.                                                                                      |
| Pode ser executado localmente? | Não como modelo aberto para instalação local; há opções de implantação empresarial pela infraestrutura da Cohere. |
| Possui API?                    | Sim.                                                                                                              |

|                  |                                                                                                                |
|------------------|----------------------------------------------------------------------------------------------------------------|
| Custo aproximado | Uso da API é cobrado conforme a quantidade de tokens processados; existe uso de teste gratuito com limitações. |
|------------------|----------------------------------------------------------------------------------------------------------------|

**Fonte:** [https://docs.cohere.com/v1/docs/models?utm\\_source](https://docs.cohere.com/v1/docs/models?utm_source)

### **Por que esse modelo é adequado ao seu cenário?**

O Cohere Embed v4.0 é adequado porque o projeto precisa realizar buscas semânticas em documentos de transporte público escritos em português. Além disso, alguns documentos podem possuir tabelas, mapas e imagens, e o modelo suporta entradas multimodais. Isso pode ser útil para documentos em PDF que contenham texto junto com imagens ou mapas.

A dimensão de 1536 também oferece uma representação detalhada dos documentos, enquanto a possibilidade de utilizar dimensões menores permite reduzir o espaço de armazenamento caso seja necessário.

### **Considerou algum modelo alternativo e descartou? Qual, e por quê?**

Um modelo alternativo seria o Cohere Embed Multilingual v3.0, que também seria uma opção adequada para o projeto por oferecer suporte a vários idiomas e permitir a criação de embeddings para textos e imagens. Porém, o Embed v4.0 foi escolhido porque apresenta recursos mais adequados ao cenário de transporte público, como suporte a textos, imagens e conteúdos mistos, incluindo PDFs, além de aceitar um contexto de até 128 mil tokens. Assim, o v3.0 seria uma alternativa viável, mas o v4.0 oferece mais recursos para trabalhar com documentos que podem conter textos, tabelas e mapas.

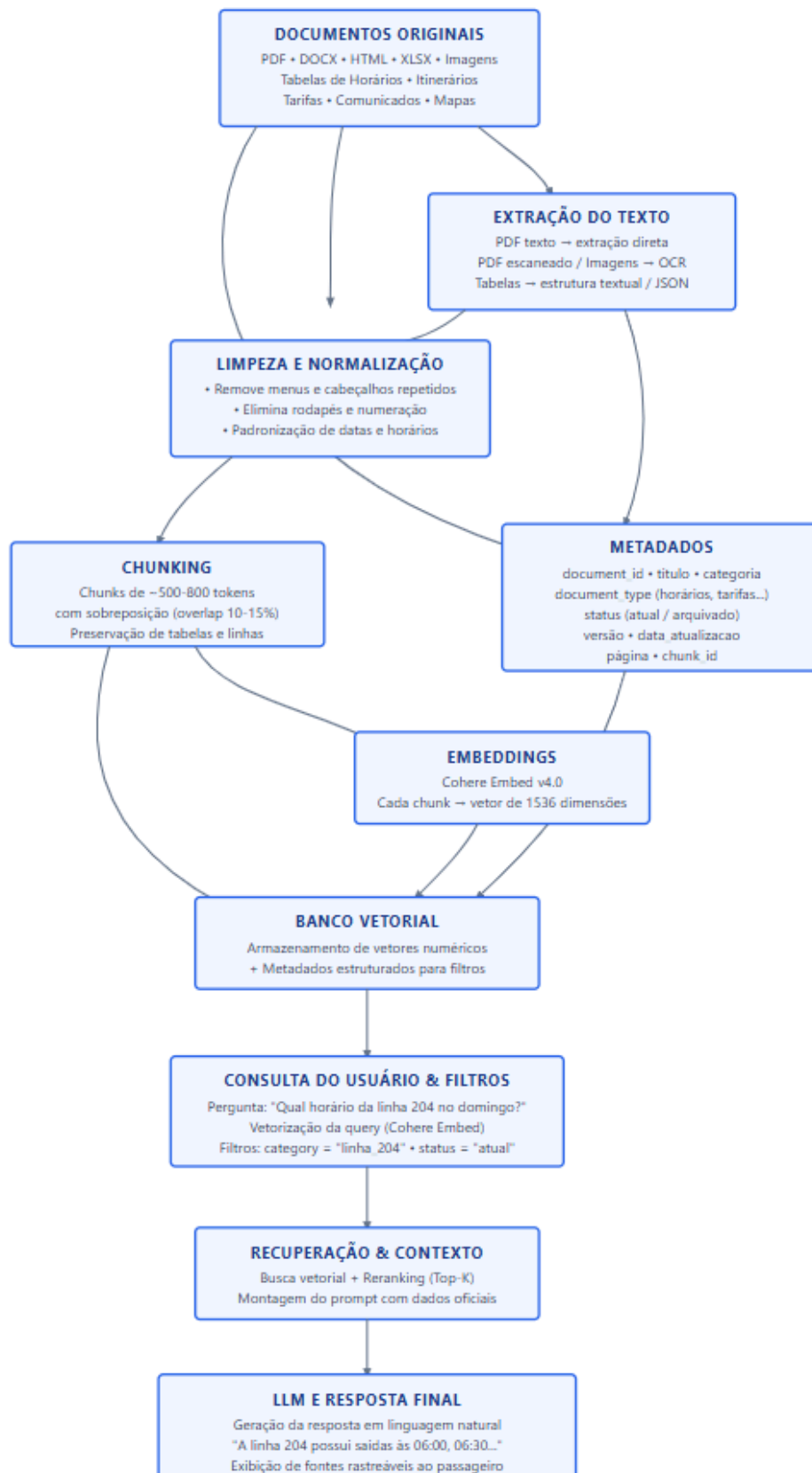
### **Se o cenário envolve documentos sigilosos, isso muda sua escolha entre modelo local e API? Como?**

Se o projeto trabalhasse com documentos sigilosos, a escolha entre API e execução local seria mais importante. Uma solução local poderia oferecer maior controle sobre os dados, evitando enviar documentos para um serviço externo. Como o cenário de transporte público utiliza principalmente informações oficiais e públicas, a utilização de uma API é uma opção mais simples e adequada.

### **O tamanho máximo de entrada do modelo tem relação com a sua decisão de chunking da Parte 5? Explique.**

Sim, existe relação. Embora o Embed v4.0 aceite até 128 mil tokens, isso não significa que devemos colocar documentos inteiros em um único embedding. No nosso projeto, continuaria utilizando chunks de aproximadamente 500 a 800 tokens, como definido anteriormente. O limite de 128 mil tokens representa a quantidade máxima de conteúdo que o modelo consegue receber de uma vez, enquanto o chunking serve para dividir os documentos em trechos menores e mais específicos, facilitando a recuperação da informação correta durante a busca.

## Arquitetura final



| Etapa      | Decisão                                                                                             | Justificativa em uma linha                                                                |
|------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Extração   | Extrair textos, tabelas e imagens; usar OCR em PDFs digitalizados.                                  | Permite aproveitar as diferentes informações presentes nos documentos de transporte.      |
| Limpeza    | Remover cabeçalhos, rodapés, páginas e espaços desnecessários, preservando informações importantes. | Evita conteúdo desnecessário sem perder horários, tarifas e alterações de linhas.         |
| Chunking   | Usar splitter recursivo com chunks de 500-800 tokens e overlap de 10-15%.                           | Melhora a precisão da busca.                                                              |
| Metadados  | Armazenar linha, tipo de documento, fonte, data, versão e status.                                   | Permite filtrar informações e evitar o uso de documentos desatualizados.                  |
| Embeddings | Utilizar o Cohere Embed v4.0                                                                        | É adequado para busca semântica, português e documentos que podem conter texto e imagens. |

### Riscos e limitações da sua própria proposta. O que você sabe que essa arquitetura não resolve bem?

**Fontes desatualizadas:** se o órgão responsável não atualizar seus documentos ou site, o sistema também poderá fornecer informações antigas.

**Alterações em tempo real:** atrasos, acidentes, obras ou mudanças de rota podem acontecer depois da última atualização da base de dados.

**Problemas nos documentos:** PDFs, tabelas e mapas podem ser difíceis de interpretar e podem causar erros durante a extração das informações.

**Perguntas ambíguas:** o usuário pode não informar o local de partida, horário ou destino, sendo necessário pedir mais informações.

**Erros do LLM:** mesmo com informações corretas recuperadas, o modelo pode interpretar ou combinar os dados de forma errada.

**Fontes conflitantes:** diferentes documentos oficiais podem apresentar horários, tarifas ou trajetos diferentes, exigindo uma forma de identificar qual informação é mais recente.

**Limitação das fontes:** se determinada informação não estiver disponível nas fontes utilizadas, o sistema não conseguirá responder com segurança.

**Consultas complexas:** encontrar a melhor rota ou combinar várias linhas pode exigir recursos adicionais, como APIs de mapas ou sistemas de roteamento.

